LAWONCALL PLATFORM

AWS Production Deployment Guide

A comprehensive, step-by-step documentation for deploying the LawMate Full-Stack application (React PWA Frontend, Fastify Microservices Gateway, and PostgreSQL Database) on Amazon Web Services. Written for both technical administrators and non-technical managers.

Status:	Production-Ready / Deployed
Last Updated:	June 10, 2026
Version:	1.1.0 (Detailed Edition)
Author:	Antigravity Dev Team

1. System Architecture Overview

The LawMate (LawOnCall) application is a decoupled, modern multi-service system. It is composed of a single-page PWA client (React/Vite) and five containerized Fastify backend microservices coordinated behind an API Gateway proxy. Native AWS services are leveraged to ensure high availability, fast response times, database backups, and SSL security.

Components and Target AWS Services:

Component	Folder / Repo	AWS Target	Purpose & Access
Client App	lawmate-pwa	S3 + CloudFront + ACM	Builds React JS/CSS files and distributes globally via S3 & CDN under HTTPS.
API Gateway	services/gateway	EC2 / ECS Fargate + ALB	Entrypoint route for all client API queries. Validates JWT and routes requests.
Auth Service	services/auth	EC2 / ECS (Internal)	Handles user sign-in, Firebase JWT verification, and email OTP codes.
Lead Service	services/lead	EC2 / ECS (Internal)	Runs automated lawyer matching logic and SLA background timers.
Payment Service	services/payment	EC2 / ECS (Internal)	Processes Razorpay & PhonePe callback webhooks and transaction states.
Profile/Notif	services/profile	EC2 / ECS (Internal)	Manages user advocate profiles, document uploads, and SMS logs.
Database	packages/db	AWS RDS PostgreSQL	Managed PostgreSQL database with automatic backups and failovers.

2. Prerequisites & Identity Management (IAM)

Before deploying, you must create a dedicated **Deployment IAM User** in the AWS Console. This IAM user provides secure access keys allowing automated GitHub Actions pipelines or developer machines to build and deploy code.

Step 2.1: Step-by-Step IAM User Creation:

1. Log in to the **AWS Management Console** (https://aws.amazon.com/console).
2. In the search bar at the top, type **IAM** and click on the first search result.

3. On the left sidebar menu, click on **Users**, then click the orange **Create user** button.
4. User Details: Set the User name to lawoncall-deployer. Keep 'Provide user access to AWS Management Console' **unchecked** (this user only needs API Access). Click **Next**.
5. Permissions Options: Select **Attach policies directly**.
6. In the search box, search and check the boxes for the following AWS managed policies:
   • AmazonEC2ContainerRegistryFullAccess (Allows pushing docker images)
   • AmazonECS_FullAccess (Allows updating running Fargate containers)
   • AmazonS3FullAccess (Allows uploading PWA client files)
   • CloudFrontFullAccess (Allows invalidating CDN cache)
7. Click **Next**, review the details, and click **Create user**.
8. Once created, click on the user name lawoncall-deployer, go to the **Security credentials** tab, scroll down to **Access keys**, and click **Create access key**.
9. Select **Command Line Interface (CLI)**, check the confirmation box, and click **Next**. Click **Create access key**.
10. **IMPORTANT:** Copy the AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY immediately, or click **Download .csv file**. Keep these keys safe. They will be used to configure environment variables.

3. Database Provisioning (AWS RDS)

AWS RDS (Relational Database Service) hosts our PostgreSQL database. This ensures automated daily backups, security patching, and scaling. Follow these step-by-step instructions to create the database:

Step 3.1: Step-by-Step RDS PostgreSQL Setup:

1. Open the search bar in the AWS Console, type **RDS**, and select the RDS service.
2. On the RDS dashboard, click the orange **Create database** button.
3. Database creation method: Select **Standard create**.
4. Engine options: Select **PostgreSQL**. Under Engine Version, select **PostgreSQL 16.1 or newer**.
5. Templates: For a production database, select **Production**. (Select **Free Tier** or **Dev/Test** if setting up a staging environment).
6. Settings: Set the **DB instance identifier** to lawoncall-prod-db. Master username: postgres. Enter a strong **Master password** and note it down safely.
7. Instance configuration: Choose **Burstable classes** and select db.t4g.micro (for staging) or db.t4g.medium (for production).
8. Storage: Allocate 20 GiB (minimum) with **gp3** storage. Keep Storage Autoscaling enabled with a threshold of 100 GiB.
9. Connectivity: Choose your target Virtual Private Cloud (VPC). Under **Public access**, select **No** (this isolates the database from public internet scanners, allowing connections only from your backend server).
10. VPC Security Group: Select **Create new**. Set the name to lawoncall-rds-sg.
11. Click **Create database**. It will take 5–10 minutes to provision the database instance.

Step 3.2: Allow Backend Connection in Security Group:

SECURITY WARNING: Never open database access port 5432 to 0.0.0.0/0 (the entire internet). This exposes the database to brute force attempts. You must restrict port 5432 inbound connections exclusively to the security group assigned to your backend EC2 instance or ECS Fargate task.

Step 3.3: Step-by-Step Security Group Rules Config:

1. In RDS console, click on your database identifier lawoncall-prod-db. Under **Connectivity & security**, click on the link under **VPC security groups**.
2. Select the security group checkbox, go to the **Inbound rules** tab at the bottom, and click **Edit inbound rules**.
3. Click **Add rule**. Set Type to **PostgreSQL (TCP Port 5432)**.

4. Source: Choose **Custom**. In the search box, select the security group of your EC2 backend server (e.g. launch-wizard-1 or lawoncall-ec2-sg) or type the VPC CIDR block (e.g. 172.31.0.0/16). Click **Save rules**.

Step 3.4: Database Schema Migration:

To initialize the tables, you must apply the Prisma schema migration. Run the following command from the root of your local repository targeting your RDS database endpoint:

```
# Set your database URL with the RDS endpoint, master username, and password

DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@rds-endpoint.amazonaws.com:5432/neondb?sslmode=requi
re"

cd packages/db && npx prisma db push
```

4. Frontend Deployment (AWS S3 + CloudFront)

The React PWA frontend (single-page application) is built into static assets (HTML, JS, CSS) and hosted on AWS S3, while CloudFront acts as a global CDN to distribute files under HTTPS with minimum latency.

Step 4.1: Build React Assets locally or via CI/CD:

1. Create an .env.production file inside the lawmate-pwa directory.
2. Paste your production environment configuration (Vite requires variables to be prefixed with VITE_):

```
VITE_API_URL=https://api.yourdomain.com
VITE_FIREBASE_API_KEY=AIzaSyA123456789...
VITE_FIREBASE_AUTH_DOMAIN=lawmate-prod.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=lawmate-prod
VITE_FIREBASE_STORAGE_BUCKET=lawmate-prod.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=8877665544
VITE_FIREBASE_APP_ID=1:8877665544:web:abcd1234ef
```

3. Open your terminal, navigate to the PWA folder, and compile the static build:

```
cd lawmate-pwa
npm install
npm run build # This outputs all static files in the lawmate-pwa/dist/ folder
```

Step 4.2: Create and Configure S3 Bucket:

1. Open S3 console and click **Create bucket**.
2. Bucket Name: Enter a unique name (e.g. lawmate-pwa-prod-bucket). Select your deployment Region.
3. Under **Block Public Access settings for this bucket**, make sure **Block all public access** is **checked**. (This ensures files are served only through the CloudFront CDN securely). Click **Create bucket**.
4. Click on the newly created bucket, click **Upload**, and drag all files and folders *inside* the local lawmate-pwa/dist/ directory into the upload console. Click **Upload**.

Step 4.3: Configure CloudFront Distribution:

1. Open CloudFront console, click **Create distribution**.
2. Origin Domain: Select your S3 bucket (e.g., lawmate-pwa-prod-bucket.s3.amazonaws.com).
3. Origin Access: Select **Origin Access Control (OAC)**. Click **Create control setting**, accept defaults, and click **Create**.
4. Viewer Protocol Policy: Select **Redirect HTTP to HTTPS**.

5. Web Application Firewall (WAF): Select **Do not enable security protections** for basic setup, or enable it for DDoS guard.
6. SSL Certificate & Domain: Under **Alternate domain name (CNAME)**, enter your domain name (e.g., www.yourdomain.com). Under **Custom SSL certificate**, select your requested ACM certificate.
7. Default Root Object: Type index.html. Click **Create distribution**.
8. **CRITICAL STEP:** Once created, copy the S3 bucket policy suggested by CloudFront, go to your S3 bucket's **Permissions** tab, scroll to **Bucket policy**, click **Edit**, and paste the policy to grant CloudFront access. Click **Save changes**.
9. **CRITICAL React Router Configuration:** In the CloudFront console under your distribution, go to the **Error pages** tab. Click **Create custom error response**. For HTTP Error Code, select **404: Not Found**. Select **Customize Error Response**. Set Response Page Path to /index.html and HTTP Response Code to **200: OK**. Click **Create**. (This ensures client-side routing handles direct URL requests without causing S3 404 errors).

5. Cost-Effective Backend EC2 Deployment

For cost-efficiency, ease of setup, and single-instance management, we deploy the Fastify Gateway API and microservices on a single AWS EC2 instance orchestrated via Docker Compose and served behind an Nginx reverse proxy with SSL.

Step 5.1: Provision EC2 Instance:

1. Open the EC2 dashboard, click **Launch instance**.
2. Instance name: lawmate-backend-server. OS: Select **Ubuntu Server 22.04 LTS**.
3. Instance Type: Select t3.medium (2 vCPUs, 4 GiB memory) to handle running all microservices smoothly.
4. Key Pair: Select an existing key pair or click **Create new key pair**. Download the .pem file and keep it secure.
5. Network Settings: Check the boxes for **Allow SSH traffic from**, **Allow HTTPS traffic from the internet**, and **Allow HTTP traffic from the internet**.
6. Click **Launch instance**.
7. **Allocate Elastic IP:** On the EC2 sidebar, select **Elastic IPs**, click **Allocate Elastic IP address**. Select the allocated IP, click **Actions** -> **Associate Elastic IP address**, and select your running lawmate-backend-server instance. (This ensures the IP address stays constant after restarts).

Step 5.2: Connect to the Server & Install Software:

1. In the EC2 instances list, select lawmate-backend-server and click the **Connect** button at the top.
2. Under the **EC2 Instance Connect** tab, click the orange **Connect** button. (This opens a secure server command line terminal directly in your web browser, removing the need for any SSH client software!).
3. Once the command prompt appears, copy and paste the following command to update packages and install Docker, Docker Compose, Nginx, and Certbot:

```
sudo apt update && sudo apt install -y docker.io docker-compose certbot python3-certbot-nginx
```

Step 5.3: Set Up Project Files:

1. Create the application folder on the EC2 server: mkdir -p /app/lawmate
2. Navigate to the folder: cd /app/lawmate
3. Create the docker-compose.yml file: nano docker-compose.yml. (Paste the configuration template shown on the next page, then press Ctrl+O to save and Ctrl+X to exit).
4. Create the .env environment file: nano .env. (Paste the variables template from Section 7, edit the placeholders with your actual passwords and keys, then save and exit).

5. Start all backend microservices: sudo docker-compose up -d. (This downloads, builds, and launches all microservices in the background). Verify status with: sudo docker-compose ps.

5.4: docker-compose.yml Production Template

This configuration launches all the LawMate microservices on the EC2 instance, enabling them to communicate internally and securely share database resources.

```
version: '3.8'

services:
  gateway:
    image: node:20-alpine
    working_dir: /app
    volumes:
      - .:/app
    command: npx tsx services/gateway/index.ts
    ports:
      - "8000:8000"
    environment:
      - PORT=8000
      - JWT_SECRET=${JWT_SECRET}
      - FRONTEND_URL=${FRONTEND_URL}
      - AUTH_SERVICE=http://auth:3001
      - LEAD_SERVICE=http://lead:3002
      - PROFILE_SERVICE=http://profile:3003
      - PAYMENT_SERVICE=http://payment:3004
      - NOTIFICATION_SERVICE=http://notification:3005
    depends_on:
      - auth
      - lead

  auth:
    image: node:20-alpine
    working_dir: /app
    volumes:
      - .:/app
    command: npx tsx services/auth/index.ts
    environment:
      - PORT=3001
      - DATABASE_URL=${DATABASE_URL}
      - FIREBASE_PROJECT_ID=${FIREBASE_PROJECT_ID}
      - FIREBASE_CLIENT_EMAIL=${FIREBASE_CLIENT_EMAIL}
      - FIREBASE_PRIVATE_KEY=${FIREBASE_PRIVATE_KEY}
```

```
  lead:
    image: node:20-alpine
    working_dir: /app
    volumes:
      - .:/app
    command: npx tsx services/lead/index.ts
    environment:
      - PORT=3002
      - DATABASE_URL=${DATABASE_URL}

  payment:
    image: node:20-alpine
    working_dir: /app
    volumes:
      - .:/app
    command: npx tsx services/payment/index.ts
    environment:
      - PORT=3004
      - DATABASE_URL=${DATABASE_URL}
      - PHONEPE_MERCHANT_ID=${PHONEPE_MERCHANT_ID}
      - PHONEPE_CLIENT_ID=${PHONEPE_CLIENT_ID}
      - PHONEPE_CLIENT_SECRET=${PHONEPE_CLIENT_SECRET}
      - PHONEPE_ENV=${PHONEPE_ENV}

  profile:
    image: node:20-alpine
    working_dir: /app
    volumes:
      - .:/app
    command: npx tsx services/profile/index.ts
    environment:
      - PORT=3003
      - DATABASE_URL=${DATABASE_URL}
```

5.5: Nginx Web Server SSL Setup (Certbot)

Nginx acts as a reverse proxy. It listens on public ports 80 (HTTP) and 443 (HTTPS), redirects all HTTP web requests to HTTPS, and forwards API queries targeting /api directly to the local Gateway running on port 8000.

Step 5.5.1: Create Nginx Site Configuration:

1. Open the Nginx config editor: sudo nano /etc/nginx/sites-available/lawoncall
2. Paste the following configuration, replacing api.yourdomain.com with your actual backend subdomain:

```
server {
    listen 80;
    server_name api.yourdomain.com;

    location / {
        proxy_pass http://localhost:8000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

3. Enable the configuration by linking it to the active sites folder:

```
sudo ln -s /etc/nginx/sites-available/lawoncall /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default # Remove default page config
sudo nginx -t # Test if syntax is correct
sudo systemctl restart nginx # Apply changes
```

Step 5.5.2: Install Free SSL Certificate via Let's Encrypt:

To enable secure HTTPS encryption, run Certbot to automatically request and install your SSL certificate. Certbot will also automatically update Nginx settings for you.

```
sudo certbot --nginx -d api.yourdomain.com
```

Follow the prompts on-screen: Enter your email, agree to the terms, and choose to **Redirect all HTTP traffic to HTTPS** automatically.

6. Environment Variables & Credentials Checklist

Make sure your production environment variables inside the /app/lawmate/.env file on the EC2 instance are populated with the correct API keys. Below is the checklist of required values:

Variable Key	Category	Value Description / Production Placeholder
DATABASE_URL	Database	postgresql://postgres:YOUR_PASSWORD@rds-endpoint.amazonaws.com:5432/neondb?sslmode=require
JWT_SECRET	Security	Any cryptographically secure random string used to sign user auth tokens.
FIREBASE_PROJECT_ID	Auth	Your Firebase Console project ID (e.g. lawmate-prod).
FIREBASE_CLIENT_EMAIL	Auth	Firebase Service Account client email address.
FIREBASE_PRIVATE_KEY	Auth	**IMPORTANT NOTICE:** Copy the service account private key. Ensure you replace all escaped newlines (\n) with actual literal newlines inside the file.
PHONEPE_MERCHANT_ID	Payment	Your production PhonePe Merchant ID.
PHONEPE_CLIENT_ID	Payment	Your PhonePe API integration Client ID.
PHONEPE_CLIENT_SECRET	Payment	Your PhonePe API Client Secret key.
PHONEPE_ENV	Payment	Set to PRODUCTION for live checkout, or SANDBOX for testing.
SMTP_HOST	Email	Mail server domain (e.g. smtp.gmail.com).
SMTP_PORT	Email	Mail port (typically 587 for TLS/STARTTLS).
SMTP_USER	Email	Email account username (e.g. support@yourdomain.com).
SMTP_PASS	Email	Email account password (use a Gmail App Password if using Gmail).

7. Troubleshooting & Verification Checklist

Use these common administrative commands on your EC2 instance terminal to manage the services and troubleshoot errors:

Action Needed	Exact Command to Run
Check microservice logs in real-time	sudo docker-compose logs -f [service_name] (e.g., sudo docker-compose logs -f gateway)
Check logs for all backend services	sudo docker-compose logs --tail=100 -f
Restart all backend microservices	sudo docker-compose restart
Apply environment variable changes	sudo docker-compose down && sudo docker-compose up -d
Check server memory and disk space usage	df -h && free -h
Check active Docker containers and resource usage	sudo docker ps && sudo docker stats
Test backend API Gateway status check	curl http://localhost:8000/api/health

8. Verification Checklist for Go-Live:

- **DNS Mapping**: Point `api.yourdomain.com` (backend) to the EC2 Elastic IP using an A Record in your registrar console (e.g. Route53, GoDaddy).
- **CloudFront SSL**: CNAME `www.yourdomain.com` points to the CloudFront distribution domain (`xxxx.cloudfront.net`).
- **Database Connectivity**: Confirm backend services show successful PostgreSQL connection logs.
- **Let's Encrypt SSL Auto-renewal**: Certbot handles renewals automatically, check cron status using `sudo systemctl status certbot.timer`.

End of Documentation.